```
    Property = await _context.Properties
        .Include(p => p.City).FirstOrDefaultAsync(m => m.Id == id);

    if (Property == null)
    {
        return NotFound();
    }
    ViewData["CityId"] = new SelectList(_context.Cities, "Id", "Id");
    return Page();
}
```

I'm not a fan of code that serves no purpose, and there is a great example of that at the beginning of this handler. The method takes a nullable parameter representing the identity of the item you want to edit. Then a block of code checks to see if a value has been passed, and it returns a 404 if not. You have a mechanism that ensures this page can *only* be reached if a number is provided. It's called *route constraints*, which you learned about in chapter 4. Instead, you can make the `id` a required route parameter of this page and constrain it to a number type, negating the need for the first code block because the framework will return a 404 if no number is provided.

This code block creates a `SelectList` populated with `City` data. The `SelectList` is assigned to `ViewData`, which, as you might recall from chapter 3, is a weakly typed dictionary. Objects retrieved from `ViewData` need to be cast to their correct type to be used again in code. Ideally, the `SelectList` should be a property of the `PageModel`, so you do not need to use casting when you use the object in the Razor page.

The code that obtains the city data is not asynchronous. As a performance best practice, you should always aim to use asynchronous APIs in an ASP.NET Core web application when they are available (http://mng.bz/epwV). Most libraries that make *out of process*, or I/O, calls, such as those that enable communication with databases, sending email, file handling, and so on, offer asynchronous APIs.